

# CKA: Mock Exams

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

---

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

## Mock Exam 1 Step by Step Solutions

---

This article provides step-by-step solutions for tasks in Mock Exam 1 related to Kubernetes configurations and operations.

In this article, we walk through the solutions for Mock Exam 1. Each section corresponds to a specific exam question and includes a technical diagram with its original description. Follow the steps carefully to complete each task.

---

### Question 1 – Create a Pod with Three Containers

---

You must create a pod named **mc-pod** in the **MC** namespace that includes three containers. You can either consult the documentation for an example pod configuration or use an imperative command with `kubectl run` and then modify the generated YAML file.

Below is an example of using `kubectl run` with the `--overrides` flag to add container definitions. The `--dry-run=client` flag outputs the YAML for further editing:

```
```bash theme={null} kubectl run mc-pod-1 --image=nginx:1-alpine --env=NODE_NAME=$(POD_NAME) --overrides=' { "apiVersion": "v1", "spec": { "containers": [ { "name": "mc-pod-2", "image": "busybox:1", "command": [ "sh", "-c", "while true; do date >> /var/log/shared/date.log; sleep 1; done" ] }, { "name": "mc-pod-3", "image": "busybox:1", "command": [ "sh", "-c", "tail -f /var/log/shared/date.log" ] } ] } }'
```

Alternatively, you can generate a base YAML file:

```
```bash theme={null}
kubectl run mc-pod --image=nginx:1-alpine --dry-run=client -o yaml > question1.yaml
```

After generating the file, update it as follows:

1. **Rename the pod:** Change its name to **mc-pod** (and remove unnecessary labels and creation timestamps).
2. **Update the first container:** Rename it to **mc-pod-1** and inject an environment variable ( `NODE_NAME` ) whose value is dynamically set from the `spec.nodeName` field. The YAML snippet becomes:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: mc-pod spec: containers: - image: nginx:1-alpine name: mc-pod-1 env: - name: NODE_NAME valueFrom: fieldRef: fieldPath: spec.nodeName resources: {}
```

1. **Define the second container:** Use the `busybox:1` image with the following command:

```
yaml theme={null} - name: mc-pod-2 image: busybox:1 command: - "sh" - "-c" - "while true; do date >> /var/log/shared/date.log; sleep 1; done"
```

1. **Define the third container:** Use the `busybox:1` image to tail the shared log file:

```
yml theme={null} - name: mc-pod-3 image: busybox:1 command: - "sh" - "-c" - "tail -f /var/log/shared/date.log"
```

1. **Add a shared volume:** Since containers 2 and 3 need to share the same filesystem, add a non-persistent volume using `emptyDir` and mount it on `/var/log/shared` for both containers. The final YAML is:

```
yml theme={null} apiVersion: v1 kind: Pod metadata: name: mc-pod spec: volumes: - name: shared-volume emptyDir: {} containers: - image: nginx:1-alpine name: mc-pod-1 env: - name: NODE_NAME valueFrom: fieldRef: fieldPath: spec.nodeName resources: {} - name: mc-pod-2 image: busybox:1 command: - "sh" - "-c" - "while true; do date >> /var/log/shared/date.log; sleep 1; done" volumeMounts: - name: shared-volume mountPath: /var/log/shared - name: mc-pod-3 image: busybox:1 command: - "sh" - "-c" - "tail -f /var/log/shared/date.log" volumeMounts: - name: shared-volume mountPath: /var/log/shared dnsPolicy: ClusterFirst restartPolicy: Always
```

After saving and applying this configuration, verify the pod's status and check the logs of container **mc-pod-3**. Its output should display the continuously appended date entries from container **mc-pod-2**.

The screenshot shows the KodeKloud interface with a task description on the left and a terminal window on the right.

**Task Description:**

Create a Pod `mc-pod` in the `mc-namespace` namespace with three containers. The first container should be named `mc-pod-1`, run the `nginx:1-alpine` image, and set an environment variable `NODE_NAME` to the node name. The second container should be named `mc-pod-2`, run the `busybox:1` image, and continuously log the output of the `date` command to the file `/var/log/shared/date.log` every second. The third container should have the name `mc-pod-3`, run the image `busybox:1`, and print the contents of the `date.log` file generated by the second container to stdout. Use a shared, non-persistent volume.

**Terminal Output:**

```
Thu Apr 24 03:27:13 UTC 2025
Thu Apr 24 03:27:14 UTC 2025
Thu Apr 24 03:27:15 UTC 2025
Thu Apr 24 03:27:16 UTC 2025
Thu Apr 24 03:27:17 UTC 2025
Thu Apr 24 03:27:18 UTC 2025
Thu Apr 24 03:27:19 UTC 2025
Thu Apr 24 03:27:20 UTC 2025
Thu Apr 24 03:27:21 UTC 2025
Thu Apr 24 03:27:22 UTC 2025
Thu Apr 24 03:27:23 UTC 2025
Thu Apr 24 03:27:24 UTC 2025
Thu Apr 24 03:27:25 UTC 2025
Thu Apr 24 03:27:26 UTC 2025
Thu Apr 24 03:27:27 UTC 2025
Thu Apr 24 03:27:28 UTC 2025
Thu Apr 24 03:27:29 UTC 2025
Thu Apr 24 03:27:30 UTC 2025
Thu Apr 24 03:27:31 UTC 2025
Thu Apr 24 03:27:32 UTC 2025
Thu Apr 24 03:27:33 UTC 2025
Thu Apr 24 03:27:34 UTC 2025
Thu Apr 24 03:27:35 UTC 2025
Thu Apr 24 03:27:36 UTC 2025
Thu Apr 24 03:27:37 UTC 2025
Thu Apr 24 03:27:38 UTC 2025
Thu Apr 24 03:27:39 UTC 2025
Thu Apr 24 03:27:40 UTC 2025
Thu Apr 24 03:27:41 UTC 2025
Thu Apr 24 03:27:42 UTC 2025
Thu Apr 24 03:27:43 UTC 2025
Thu Apr 24 03:27:44 UTC 2025
Thu Apr 24 03:27:45 UTC 2025
Thu Apr 24 03:27:46 UTC 2025
Thu Apr 24 03:27:47 UTC 2025
Thu Apr 24 03:27:48 UTC 2025
Thu Apr 24 03:27:49 UTC 2025
```

## Question 2 – Prepare Node One for Kubernetes

For this task, you will perform several actions on node one.

1. **SSH into node one** using the provided credentials:

```
bash theme={null} ssh node01 # Accept the host key and provide the password when prompted.
```

1. **Switch to the root user** (if necessary):

```
bash theme={null} sudo su
```

1. **Navigate to the `/root` directory** and locate the CRI Docker package (e.g., `cri-docker_0.3.16.3-0.debian.deb`).

2. **Install the CRI Docker package:**

```
bash theme={null} dpkg -i ./cri-docker_0.3.16.3-0.debian.deb
```

1. **Start the CRI Docker service:**

```
bash theme={null} systemctl start cri-docker
```

1. **Verify the service status:**

```
bash theme={null} systemctl status cri-docker
```

1. **Enable the service to start on boot:**

```
bash theme={null} systemctl enable cri-docker
```

Ensure that the service is active (running) and enabled.

If you encounter any issues with the service, recheck the installation and confirm your package file's integrity.

---

## Question 3 – Save Vertical Pod Autoscaler CRDs

On the control plane node, list all Custom Resource Definitions (CRDs) related to the Vertical Pod Autoscaler and save their names to `/root/vpa-crds.txt`.

1. **Retrieve all CRDs and filter for “vertical”:**

```
bash theme={null} kubectl get crd | grep -i vertical
```

Expected output:

```
text      theme={null}    verticalpodautoscalercheckpoints.autoscaling.k8s.io    2025-04-24T03:13:05Z
verticalpodautoscalers.autoscaling.k8s.io 2025-04-24T03:13:05Z
```

1. **Save the CRD names** to a file named `/root/vpa-crds.txt`. The file should include:

```
text      theme={null}    verticalpodautoscalercheckpoints.autoscaling.k8s.io
verticalpodautoscalers.autoscaling.k8s.io
```

No further modifications are required.

---

## Question 4 – Create a Service for the Messaging Application

Expose the messaging application by creating a service named **messaging-service** on port **6379** within the cluster.

1. **Verify that the messaging pod is running:**

```
bash theme={null} kubectl get pod
```

### 1. Expose the messaging pod:

```
bash theme={null} kubectl expose pod messaging --port=6379 --name=messaging-service
```

### 1. Confirm the service creation by describing it:

```
bash theme={null} kubectl describe service messaging-service
```

The output should list the service as a ClusterIP service on port **6379** with endpoints matching the messaging pod's IP.

---

## Question 5 – Create a Deployment for the HR Web Application

Create a deployment named **hr-web-app** using the image `kodekloud/webapp-color` with two replicas.

### 1. Run the following command:

```
bash theme={null} kubectl create deployment hr-web-app --image=kodekloud/webapp-color --replicas=2
```

### 1. Verify the deployment status:

```
bash theme={null} kubectl get deployment hr-web-app
```

Ensure that both replicas become available.

---

## Question 6 – Fix the Faulty Init Container in the Orange Pod

The pod named **orange** is failing because its init container is crashing with exit code 127 due to a typo in the command ( `sleep` instead of `sleeep` ).

### 1. Inspect the pod logs to identify the error:

```
bash theme={null} kubectl logs orange -c init-myservice
```

Expected error output:

```
text theme={null} sh: sleeep: not found
```

### 1. Retrieve the pod configuration and save it to a file:

```
bash theme={null} kubectl get pod orange -o yaml > question6.yaml
```

### 1. Edit the file `question6.yaml` to correct the command in the init container. Modify the snippet to:

```
yaml theme={null} initContainers: - name: init-mysvc image: busybox command: - "sh" - "-c" - "sleep 2"
```

### 1. Force update the pod with the corrected configuration:

```
bash theme={null} kubectl replace -f question6.yaml --force
```

1. **Verify the pod status** to ensure it transitions to a running state and the init container completes successfully.
- 

## Question 7 – Expose HR Web App via a NodePort Service

Expose the **hr-web-app** deployment as a service named **hr-web-app-service** to make it accessible on port **30082** from the nodes. Note that the web application listens internally on port **8080**.

1. **Generate a NodePort service configuration with dry-run:**

```
bash theme={null} kubectl expose deployment hr-web-app --type=NodePort --port=8080 --name=hr-web-app-service --dry-run=client -o yaml > question7.yaml
```

1. **Edit `question7.yaml`** to include the `nodePort: 30082` under the ports section as shown below:

```
yaml theme={null} apiVersion: v1 kind: Service metadata: name: hr-web-app-service labels: app: hr-web-app spec: type: NodePort selector: app: hr-web-app ports: - port: 8080 targetPort: 8080 nodePort: 30082
```

1. **Apply the updated service configuration:**

```
bash theme={null} kubectl apply -f question7.yaml
```

1. **Verify the service details:**

```
bash theme={null} kubectl describe service hr-web-app-service
```

---

## Question 8 – Create a Persistent Volume

Create a persistent volume named **pv-analytics** with the following specifications:

- **Capacity:** 100Mi
- **Access Mode:** ReadWriteMany
- **Type:** hostPath using the directory `/pv/data-analytics`
- **Create a YAML file (e.g., `question8.yaml`)** with the following content:

```
yaml theme={null} apiVersion: v1 kind: PersistentVolume metadata: name: pv-analytics spec: capacity: storage: 100Mi volumeMode: Filesystem accessModes: - ReadWriteMany hostPath: path: /pv/data-analytics
```

1. **Apply the configuration:**

```
bash theme={null} kubectl apply -f question8.yaml
```

1. **Verify that the persistent volume is available:**

```
bash theme={null} kubectl get pv
```

---

## Question 9 – Create a Horizontal Pod Autoscaler (HPA)

Create an HPA for the deployment **kkapp-deploy** in the default namespace with the following configuration:

- **CPU Utilization:** Maintain an average of 50%
- **Replicas:** Scale between 2 and 10 pods
- **Stabilization Window:** 300 seconds when scaling down
- **Prepare a YAML file (e.g., `webapp-hpa.yaml` )** with this content:

```
yaml theme={null} apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: webapp-hpa namespace: default spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: kkapp-deploy minReplicas: 2 maxReplicas: 10 metrics: - type: Resource resource: name: cpu target: type: Utilization averageUtilization: 50 behavior: scaleDown: stabilizationWindowSeconds: 300
```

1. **Apply the HPA configuration:**

```
bash theme={null} kubectl apply -f webapp-hpa.yaml
```

1. **Check the HPA status:**

```
bash theme={null} kubectl get hpa
```

---

## Question 10 – Create a Vertical Pod Autoscaler (VPA)

Create a Vertical Pod Autoscaler that automatically adjusts CPU and memory requests for the deployment **analytics-deployment** in the default namespace. The VPA should run in auto mode.

1. **Create a YAML file (e.g., `question10.yaml` )** with the following content:

```
yaml theme={null} apiVersion: autoscaling.k8s.io/v1 kind: VerticalPodAutoscaler metadata: name: analytics-vpa namespace: default spec: targetRef: apiVersion: apps/v1 kind: Deployment name: analytics-deployment updatePolicy: updateMode: "Auto"
```

1. **Apply the VPA configuration:**

```
bash theme={null} kubectl apply -f question10.yaml
```

1. **Verify that the VPA has been created:**

```
bash theme={null} kubectl get vpa
```

---

## Question 11 – Create a Kubernetes Gateway Resource

Create a Gateway resource for the web application.

1. **Create a YAML file (e.g., `question11.yaml` )** with the following content:

```
yaml theme={null} apiVersion: gateway.networking.k8s.io/v1 kind: Gateway metadata: name: web-gateway namespace: nginx-gateway spec: gatewayClassName: nginx listeners: - name: http protocol: HTTP port: 80
```

### 1. Apply the gateway configuration:

```
bash theme={null} kubectl apply -f question11.yaml
```

### 1. Verify that the Gateway resource is available in the `nginx-gateway` namespace:

```
bash theme={null} kubectl get gateway -n nginx-gateway
```

---

## Question 12 – Update a Helm Chart Deployment

---

A coworker deployed an NGINX Helm chart called **kk-dash-mock-one** in the **kk-dash-ns** namespace. An update to the chart is available, and you need to update the Helm repository and upgrade the release to version **18.1.15**.

### 1. List the current Helm releases in the concerned namespace:

```
bash theme={null} helm list -n kk-ns
```

### 1. Check your chart repositories:

```
bash theme={null} helm repo list
```

### 1. Update all repositories to fetch the latest chart versions:

```
bash theme={null} helm repo update
```

### 1. Search the repository for available versions of the NGINX chart:

```
bash theme={null} helm search repo nginx --versions
```

### 1. Upgrade the release with the specified chart version:

```
bash theme={null} helm upgrade kk-mock1 kk-mock1/nginx --version 18.1.15 -n kk-ns
```

### 1. Verify the upgrade by listing the release again:

```
bash theme={null} helm list -n kk-ns
```

---

By following these detailed steps and verifying each component with the respective `kubectl` and `helm` commands, you will successfully complete all the tasks for Mock Exam 1.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Mock Exam 2 Step by Step Solutions

This article provides step-by-step solutions for Kubernetes cluster configuration and troubleshooting across various scenarios.

This article provides detailed step-by-step solutions for Mock Exam Two. Each question covers a key aspect of Kubernetes cluster configuration—from creating storage classes and deployments to configuring ingress, RBAC, network policies, HPA setups, and troubleshooting common issues. Follow the solutions below to configure your clusters and gain hands-on experience with Kubernetes.

## Question 1 – Creating a Default Local StorageClass

In this question, you will create a StorageClass named **local-sc** on Cluster One's control plane. This StorageClass must be set as the default with the following criteria:

1. SSH into your control plane:

```
``plaintext theme={null} ssh cluster1-controlplane Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-1078-gcp
x86_64) * Documentation: https://help.ubuntu.com/ * Management: https://landscape.canonical.com * Support:
https://ubuntu.com/pro
```

This system has been minimized by removing packages and content that are not required on a system that users do not log into. To restore this content, you can run the 'unminimize' command. ``

1. Referencing the documentation, copy the example configuration and update it as follows:
2. Change the `name` to **local-sc**.
3. Update the annotation `storageclass.kubernetes.io/is-default-class` to `"true"`.
4. Set the `provisioner` to `kubernetes.io/no-provisioner`.
5. Remove any unnecessary fields (such as `reclaimPolicy` and `mountOptions`) not specified.
6. Retain only `allowVolumeExpansion` and `volumeBindingMode: WaitForFirstConsumer`.

The final YAML configuration should look like this:

```
``yaml theme={null} apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: local-sc annotations:
storageclass.kubernetes.io/is-default-class: "true" provisioner: kubernetes.io/no-provisioner
allowVolumeExpansion: true volumeBindingMode: WaitForFirstConsumer
```

After saving the file (e.g., as `question1.yaml`), apply the configuration:

```
```bash theme={null}
kubectl apply -f question1.yaml
```

You should see the following confirmation:

```
```plaintext theme={null} storageclass.storage.k8s.io/local-sc created
```

\*\*\*

## ## Question 2 – Deployment with App and Sidecar Containers for Logging

This question guides you through creating a deployment named **logging-deployment** in the `logging-ns` namespace.

\* **app-container**: Uses the busybox image to continuously append log entries to `/var/log/app/app.log`.

\* **log-agent**: Also based on busybox, this container tails the same log file.

Both containers share an `emptyDir` volume mounted at `/var/log/app`.

Below is the complete YAML configuration (save as `question2.yaml`):

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: logging-deployment
  namespace: logging-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: logger
  template:
    metadata:
      labels:
        app: logger
    spec:
      volumes:
        - name: log-volume
          emptyDir: {}
      containers:
        - name: app-container
          image: busybox
          command:
            - sh
            - -c
            - "while true; do echo 'Log entry' >> /var/log/app/app.log; sleep 5; done"
          volumeMounts:
            - name: log-volume
              mountPath: /var/log/app
        - name: log-agent
          image: busybox
          command:
            - tail
            - -f
            - /var/log/app/app.log
```
```

Apply the configuration:

```
```bash theme={null} kubectl apply -f question2.yaml
```

Next, verify that the pod is running and inspect its logs:

```
```bash theme={null}
kubectl get pod -n logging-ns
kubectl logs <pod-name> -c log-agent -n logging-ns
```

You should see repeated "Log entry" outputs confirming that the log file is being written and tailed properly.

---

## Question 3 – Creating an Ingress Resource to Route Traffic

Here, you will create an Ingress resource named **webapp-ingress** in the `ingress-ns` namespace. This ingress routes traffic to a service called **webapp-svc**, using the NGINX ingress controller with the following criteria:

- Hostname: `kodekloud-ingress.app`
- Path: `/` (with `PathType: Prefix`)
- Forward traffic to **webapp-svc** on port 80

Below is the YAML configuration (save as `question3.yaml`):

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: webapp-ingress
namespace: ingress-ns annotations: nginx.ingress.kubernetes.io/rewrite-target: / spec: ingressClassName: nginx
rules: - host: kodekloud-ingress.app http: paths: - path: / pathType: Prefix backend: service: name: webapp-svc
port: number: 80
```

Apply the configuration:

```
```bash theme={null}
kubectl apply -f question3.yaml
```

Test the ingress setup by sending an HTTP request:

```
```bash theme={null} curl -s http://kodekloud-ingress.app/
```

You should see the default NGINX welcome page, confirming correct configuration.

\*\*\*

### ## Question 4 – Updating an Nginx Deployment with Rolling Updates

In this question, create a deployment named **nginx-deploy** using **nginx:1.16** and then perform a rolling update.

1. Generate an initial deployment YAML using a dry run:

```
```bash theme={null}
kubectl create deployment nginx-deploy --image=nginx:1.16 --dry-run=client -o yaml > question4.yaml
```
```

The generated YAML will be similar to:

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  labels:
    app: nginx-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-deploy
  template:
    metadata:
      labels:
        app: nginx-deploy
    spec:
      containers:
        - name: nginx
          image: nginx:1.16
          resources: {}
```
```

2. Apply the deployment:

```
```bash theme={null}
kubectl apply -f question4.yaml
```
```

3. Update the image to version 1.17 using a rolling update:

```
```bash theme={null}
kubectl set image deployment/nginx-deploy nginx=nginx:1.17
```
```

4. Verify the rollout history:

```
```bash theme={null}
kubectl rollout history deployment nginx-deploy
kubectl rollout history deployment nginx-deploy --revision=2
```
```

The output should indicate that revision 2 is using **\*\*nginx:1.17\*\***.

\*\*\*

## ## Question 5 – Creating a User via CSR and Configuring RBAC

This question has two parts:

### ### Part A: Create a CertificateSigningRequest (CSR) for User "john"

1. The private key is located at ``/root/cka/john.key`` and the CSR file at ``/root/cka/john.csr``.
2. The CSR object should use the signer ``kubernetes.io/kube-apiserver-client`` and specify these

Example CSR YAML (save as ``question5.yaml``):

```
```yaml theme={null}
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: john-developer
spec:
  signerName: kubernetes.io/kube-apiserver-client
  request: <BASE64_ENCODED_CONTENT_OF_/root/cka/john.csr>
  usages:
    - digital signature
    - key encipherment
    - client auth
```

To obtain the Base64 encoded CSR, run:

```
```bash theme={null} cat /root/cka/john.csr | base64 | tr -d '\n'
```

Apply the CSR:

```
```bash theme={null}
kubectl apply -f question5.yaml
```

Approve the CSR:

```
```bash theme={null} kubectl certificate approve john-developer
```

### ### Part B: Grant RBAC Permissions with Role and RoleBinding

Create a Role named **developer** in the `development` namespace to allow the following verbs on  `pods` :

Save the following YAML as `question5-rbac.yaml`:

```
```yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
  namespace: development
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["create", "list", "get", "update", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: john-developer-role-binding
  namespace: development
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

Apply the RBAC configuration:

```
```bash theme={null} kubectl apply -f question5-rbac.yaml
```

Test the permissions by running:

```
```bash theme={null}
kubectl auth can-i create pods --as=john -n development
```

A response of **yes** confirms that the RBAC configuration is working.

---

## Question 6 – Creating an Nginx Pod with an Internal Service and DNS Testing

In this exercise, you will deploy an Nginx pod named **nginx-resolver** and expose it internally using a ClusterIP service named **nginx-resolver-service**. Then you will verify DNS resolution using a BusyBox pod.

1. Create the Nginx pod:

```
bash theme={null} kubectl run nginx-resolver --image=nginx
```

1. Expose the pod internally:

```
bash theme={null} kubectl expose pod nginx-resolver --name=nginx-resolver-service --port=80 --target-port=80 --type=ClusterIP
```

1. Verify the service endpoints:

```
bash theme={null} kubectl get svc nginx-resolver-service kubectl get pod -o wide
```

1. Run a temporary BusyBox pod to perform an nslookup:

```
bash theme={null} kubectl run test-nslookup --image=busybox:1.28 --rm -it --restart=Never --nslookup nginx-resolver-service
```

Optionally, redirect the DNS lookup results to a file (e.g., `/root/cka/nginx.svc`) if necessary. Remember that Kubernetes creates DNS entries for both pods and services.

---

## Question 7 – Creating a Static Pod

Create a static pod named **nginx-critical** on Node One. This pod should automatically restart on failure and must reside in the `/etc/kubernetes/manifests` directory.

1. Generate a pod YAML definition via dry run:

```
bash theme={null} kubectl run nginx-critical --image=nginx --restart=Always --dry-run=client -o yml > static.yml
```

1. SSH into Node One:

```
bash theme={null} ssh cluster1-node01
```

1. Place the YAML file into `/etc/kubernetes/manifests/static.yml` with contents similar to:

```
yml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx-critical labels: run: nginx-critical spec: containers: - name: nginx-critical image: nginx dnsPolicy: ClusterFirst restartPolicy: Always
```

The kubelet will automatically create the static pod. Verify its status:

```
```bash theme={null} kubectl get pod -o wide
```

```
***
```

### ## Question 8 – Creating a Horizontal Pod Autoscaler (HPA)

Create an HPA for a deployment named **backend-deployment** in the `backend` namespace. The HPA should target the deployment.

Below is the example HPA YAML (save as `webapp-hpa.yaml`):

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: backend
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-deployment
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 65
```
```

Apply the HPA configuration:

```
```bash theme={null} kubectl apply -f webapp-hpa.yaml
```

Verify the HPA details:

```
```bash theme={null}
kubectl describe hpa -n backend
```

---

## Question 9 – Troubleshooting a Non-Responsive API Server

If Cluster Two fails to respond to `kubectl` commands (e.g., with the error message "The connection to the server cluster2-controlplane:6443 was refused"), perform the following steps:

1. Run a command such as:

```
bash theme={null} kubectl get nodes
```

which may yield:

```
plaintext theme={null} The connection to the server cluster2-controlplane:6443 was refused - did you specify the right host or port?
```

1. Use `crictl pods` to check the running containers and notice that the API server container is missing.

## 2. Verify the kubelet status:

```
bash theme={null} sudo systemctl status kubelet
```

If it is inactive, start and enable the kubelet:

```
bash theme={null} sudo systemctl start kubelet sudo systemctl enable kubelet
```

After starting the kubelet, the API server container should be recreated. Confirm by running:

```
``bash theme={null} kubectl get nodes
```

```
***
```

## ## Question 10 – Modifying the Web Gateway for HTTPS

Modify the existing web gateway in the `cka5673` namespace on Cluster One so that it handles HTTPS traffic.

1. Verify that the secret exists:

```
```bash theme={null}
kubectl get secret -n cka5673
```
```

2. Retrieve the current gateway configuration:

```
```bash theme={null}
kubectl get gateway -n cka5673 -o yaml > question10.yaml
```
```

3. Edit the YAML file to update the listener section. Change to the following:

```
```yaml theme={null}
listeners:
- name: https
  port: 443
  protocol: HTTPS
  hostname: kodekloud.com
  tls:
    certificateRefs:
    - name: kodekloud-tls
```
```

4. Apply the updated configuration:

```
```bash theme={null}
kubectl apply -f question10.yaml
```
```

```
***
```

## ## Question 11 – Uninstalling a Vulnerable Helm Release

Identify and uninstall the Helm release that uses the vulnerable image `kodekloud/webapp-color:v1`.

1. List all Helm releases across namespaces:

```
```bash theme={null}
helm list -A
```
```

2. Search for the vulnerable image in each release's manifests:

```
```bash theme={null}
helm get manifest <release-name> -n <namespace> | grep -i kodekloud/webapp-color:v1
```
```

3. Once identified (for example, if the release is named `atlanta-page-apd` in the `atlanta-page-04` namespace,

```
```bash theme={null}
helm uninstall atlanta-page-apd -n atlanta-page-04
```
```

This action removes the vulnerable release from your cluster.

\*\*\*

## ## Question 12 – Applying a Network Policy

Implement a network policy that allows traffic from frontend applications (namespace `frontend`) to backend

After reviewing the provided policies, **net-pol-3.yaml** is the correct choice. Its contents are as follows:

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: net-policy-3
  namespace: backend
spec:
  podSelector: {}
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            name: frontend
  ports:
    - protocol: TCP
      port: 80
```
```

Apply the policy without affecting any other existing policies:

```
bash theme={null} kubectl apply -f net-pol-3.yaml
```

## Question 13 – Troubleshooting a Failed Deployment Due to Resource Quota

On Cluster Three, if the **backend-api** deployment fails to scale to three replicas due to resource quota limitations, follow these troubleshooting steps:

1. Describe the deployment to see error events:

```
bash theme={null} kubectl describe deployment backend-api
```

Also check the ReplicaSets for error events indicating that pod creation is forbidden by resource quotas.

1. Describe the resource quota:

```
bash theme={null} kubectl describe resourcequota cpu-mem-quota
```

An example output might be:

```
```plaintext theme={null} Name: cpu-mem-quota Namespace: default Resource Used Hard
```

```
requests.cpu 200m 300m requests.memory 256Mi 300Mi ``
```

If the new pod's request (e.g., 128Mi) exceeds the quota, adjustments are needed.

1. Update the deployment's resource requests to ensure the total for three replicas remains within the limits. For instance, reduce the memory request from `128Mi` to `100Mi`:

```
yaml theme={null} resources: requests: cpu: 100m memory: 100Mi limits: cpu: 150m memory: 150Mi
```

1. Update the deployment using editing or applying a modified YAML, and if necessary, delete the problematic ReplicaSet to trigger a new rollout:

```
bash theme={null} kubectl edit deployment backend-api kubectl delete rs <offending-replicaset>
```

After the changes, all three pods should start and run successfully.

## Question 14 – Deploying Calico CNI with a Custom CIDR

On Cluster Four, deploy Calico as your CNI provider. Use the official Calico custom-resources YAML from GitHub and modify the CIDR to `172.17.0.0/16`.

1. Download the custom-resources file:

```
bash theme={null} curl -O https://raw.githubusercontent.com/projectcalico/calico/v3.29.2/manifests/custom-resources.yaml
```

1. Edit the downloaded YAML file. Locate the Calico IP pool definition and update the `cidr` value as shown below:

```
yaml theme={null} apiVersion: operator.tigera.io/v1 kind: Installation metadata: name: default
spec: calicoNetwork: ipPools: - name: default-ip4-ipool blockSize: 26 cidr: 172.17.0.0/16
encapsulation: VXLANCrossSubnet natOutgoing: Enabled nodeSelector: all
```

1. Apply the modified configuration:

```
bash theme={null} kubectl apply -f custom-resources.yaml
```

1. To verify that Calico is operating correctly and pod-to-pod communication works, deploy a test pod (such as an Nginx pod) and use a BusyBox or net-tools container for connectivity tests:

```
bash theme={null} kubectl run web-app --image=nginx kubectl run test --rm -it --image=jrcs/net-
tools --restart=Never -- sh -c "curl -s <web-app-pod-IP>"
```

If the connectivity test is successful, Calico is configured and running correctly.

This article provides a comprehensive walkthrough of essential Kubernetes configurations and troubleshooting steps. By following these step-by-step solutions, you can better understand Kubernetes components and improve your operational skills.

Happy learning and good luck on your Kubernetes journey!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

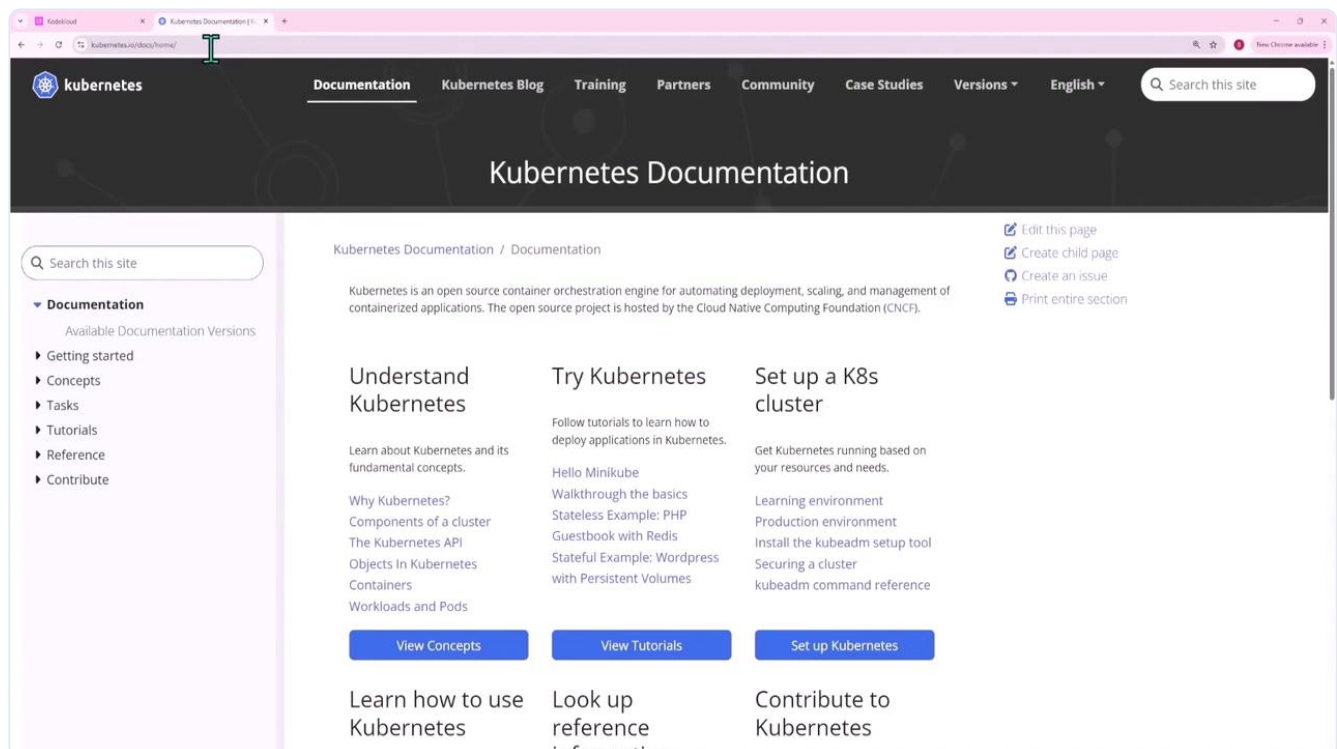
# Mock Exam 3 Step by Step Solutions

This lesson provides detailed solutions for each question in Mock Exam Three, focusing on specific Kubernetes tasks with clear instructions and code examples.

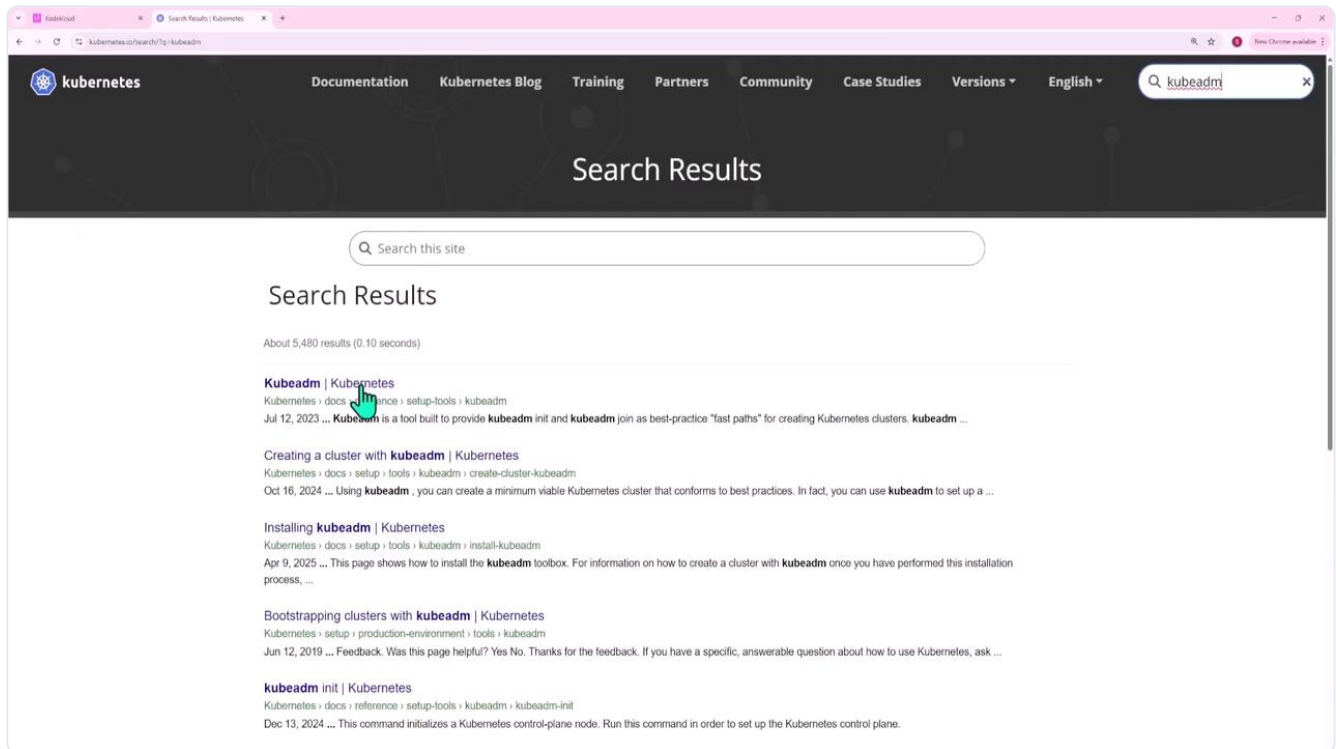
This lesson presents detailed solutions for each question in Mock Exam Three. Each solution focuses on a specific Kubernetes task and provides clear instructions, configuration code blocks, and diagram references. All image links and descriptions remain exactly as provided.

## Question 1 – Adjusting Network Parameters for Kubernetes

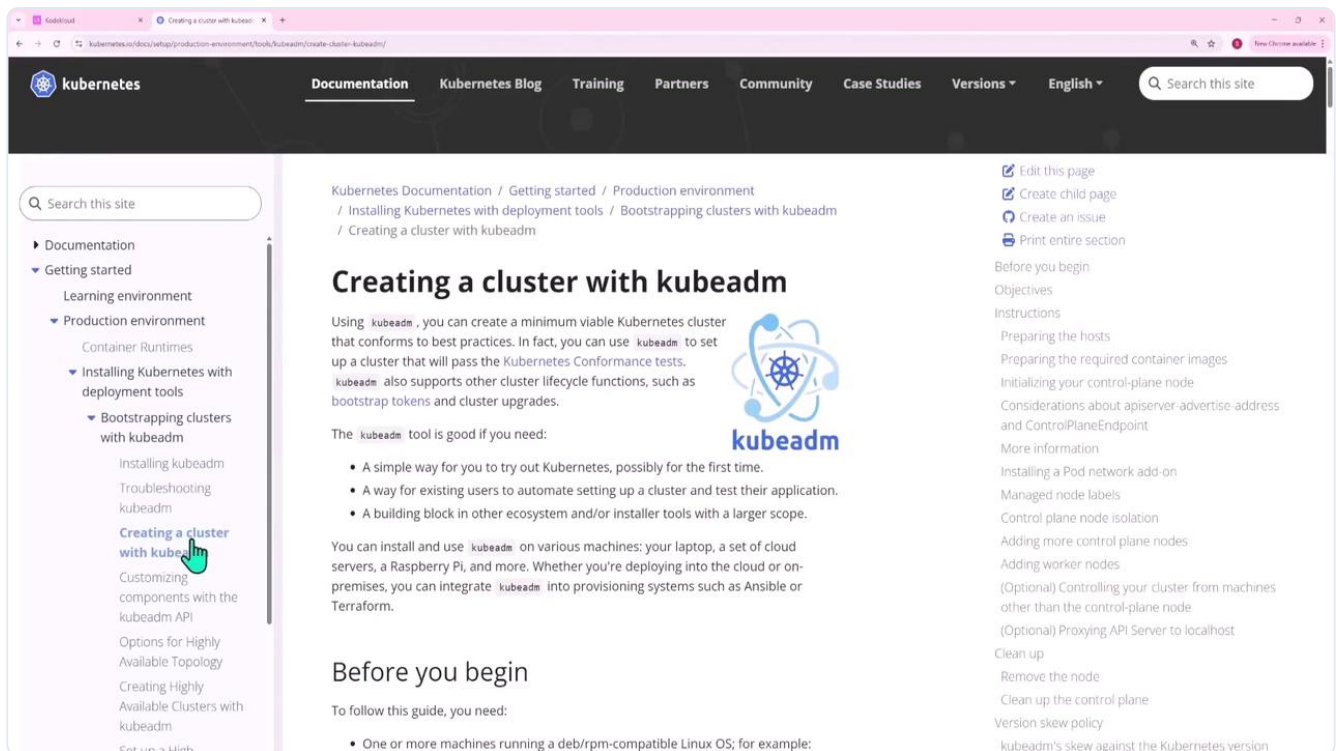
To deploy a Kubernetes cluster using kubeadm, you must enable IPv4 packet forwarding and ensure the settings persist across reboots. Refer to the kubeadm documentation for guidance when provisioning a new cluster.



Searching for “kubeadm” in the docs will help you locate the bootstrapping guide.



Navigate through the following path: Production Environment → Installing Kubernetes Deployment Tools → Bootstrapping a Cluster → Creating a Cluster with kubeadm.



The first step is to set up a container runtime and enable IPv4 packet forwarding using these commands:

```
``bash theme={null} cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf net.ipv4.ip_forward = 1 EOF
```

```
sudo sysctl --system
```

```
sysctl net.ipv4.ip_forward
```

For additional persistence, use this command if provided:

```
```bash theme={null}
# sysctl params required by setup, params persist across reboots
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip.forward = 1
EOF

# Apply sysctl params without reboot
sudo sysctl --system
```

Always copy the exact command names from the exam instructions to avoid errors.

This completes Question 1.

---

## Question 2 – Creating a Service Account and Granting PVC Listing Permissions

---

In this question you will:

1. Create a service account named **pvviewer**.
2. Create a cluster role (**pvviewer-role**) that grants permission to list persistent volumes.
3. Bind the role to the service account with a cluster role binding (**pvviewer-role-binding**).
4. Launch a pod (**pvviewer**) using the Redis image in the default namespace.

### Step 1: Create the Service Account

```
```bash theme={null} kubectl create serviceaccount pvviewer kubectl get sa
```

\*Expected output:\*

```
```plaintext theme={null}
NAME          SECRETS  AGE
default       0        6m55s
pvviewer      0        5s
```

### Step 2: Create the Cluster Role

Create the role with the required permission:

```
```bash theme={null} kubectl create clusterrole pvviewer-role --resource=persistentvolumes --verb=list
```

Verify with:

```
```bash theme={null}
kubectl describe clusterrole pvviewer-role
```

*Expected output snippet:*

```
```plaintext theme={null} Name: pvviewer-role Labels: Annotations: PolicyRules: Resource Non-Resource URLs
Resource Names Verbs
```

---

```
persistentvolumes [] [] [list]
```

```
### Step 3: Bind the Role to the Service Account
```

```
```bash theme={null}
kubectl create clusterrolebinding pvviewer-role-binding --clusterrole=pvviewer-role --serviceaccount=default
```

## Step 4: Launch the Pod

Create a pod manifest (e.g., `question2.yaml`):

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: pvviewer spec: serviceAccountName: pvviewer
containers: - name: pvviewer image: redis
```

Apply the manifest:

```
```bash theme={null}
kubectl apply -f question2.yaml
```

Verify the pod and its service account:

```
```bash theme={null} kubectl get pod kubectl describe pod pvviewer
```

This completes Question 2.

```
<Frame>
  ![The image shows a Kubernetes task description on the left, instructing to create a service account and
</Frame>
```

```
***
```

## ## Question 3 – Creating a Storage Class

Create a storage class called `**rancher-sc**` with these settings:

```
* Provisioner: `rancher.io/local-path`
* Allow volume expansion: `true`
* Volume binding mode: `WaitForFirstConsumer`
```

Example manifest (``question3.yaml``):

```
```yaml theme={null}
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: rancher-sc
provisioner: rancher.io/local-path
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

Apply the storage class:

```
```bash theme={null} kubectl apply -f question3.yaml
```

This completes Question 3.

<Frame>

![The image shows a search results page from the Kubernetes website, displaying results for the query]

</Frame>

\*\*\*

## ## Question 4 - Configuring a ConfigMap and Updating a Deployment

In the **cm-namespace**, perform these tasks:

1. Create a ConfigMap **app-config** containing key-value pairs such as `ENV=production` and `LOG_LEVEL=info`
2. Update the existing deployment **cm-web-app** to source environment variables from the ConfigMap.

### ### Step 1: Create the ConfigMap

```
```bash theme={null}
kubectl create configmap app-config -n cm-namespace --from-literal=ENV=production --from-literal=LOG_LEVEL=info
```

Verify with:

```
```bash theme={null} kubectl describe cm app-config -n cm-namespace
```

### ### Step 2: Update the Deployment

Edit the deployment to include the ConfigMap:

```
```bash theme={null}
kubectl edit deployment cm-webapp -n cm-namespace
```

Add the following under the container section:

```
```yaml theme={null} envFrom: - configMapRef: name: app-config
```

After saving, verify that new pods include the environment variables from **app-config**.

This completes Question 4.

<Frame>

! [The image shows a Kubernetes task interface with instructions to create a ConfigMap and modify a Deployment.]

</Frame>

\*\*\*

## ## Question 5 - Configuring Priority Classes and Pod Priority

For this task, you need to:

1. Create a PriorityClass **low-priority** with a value of 50,000.
2. Modify the existing pod **lp-pod** (in the **low-priority** namespace) to reference this PriorityClass.
3. Recreate the pod so that it picks up the new priority without manually setting a numeric value.

### ### Step 1: Create the PriorityClass

Create a manifest (e.g., `question5.yaml`):

```
```yaml
theme: {null}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 50000
globalDefault: false
description: "This is a low priority class"
```
```

Apply it:

```
```bash
theme: {null}
kubectl apply -f question5.yaml
```
```

### ### Step 2: Update the Pod Manifest

Create or edit the pod manifest (e.g., `question5-pod.yaml`) to include only the `PriorityClassName`:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: lp-pod
  namespace: low-priority
  labels:
    run: lp-pod
spec:
  priorityClassName: low-priority
  containers:
    - name: lp-pod
      image: nginx
      imagePullPolicy: Always
      resources: {}
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

Do not include a numeric `priority` field.

Replace the pod if needed:

```
```bash theme={null} kubectl replace -f question5-pod.yaml --force
```

## If an error appears about a numeric priority, remove any "priority: 0" specification and apply again:

---

```
kubectl apply -f question5-pod.yaml
```

Finally, verify the pod:

```
```bash theme={null}
kubectl get pod -n low-priority
```

This completes Question 5.

---

## Question 6 – Fixing Incoming Connection Issues with a Network Policy

---

A pod (**np-test-1**) and its service (**np-test-service**) are not receiving incoming traffic on port 80. Create a NetworkPolicy named **test-network-policy** to allow TCP traffic on port 80.

First, confirm the pod's labels:

```
```bash theme={null} kubectl get pod --show-labels
```

Then, create a manifest (e.g., `question6.yaml`):

```
```yaml theme={null}
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      run: np-test-1
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - protocol: TCP
          port: 80
```

Apply the policy:

```
```bash theme={null} kubectl apply -f question6.yaml
```

This policy permits incoming TCP traffic on port 80 for pods labeled `run=np-test-1`.

\*\*\*

### ## Question 7 – Tainting a Node and Creating Pods with Tolerations

In this question, you will:

1. Taint a worker node (**node01**) with `env\_type=production:NoSchedule`.
2. Create a pod (**dev-redis**) without tolerations so it avoids node01.
3. Create another pod (**prod-redis**) with a toleration to allow scheduling on node01.

#### ### Step 1: Taint the Node

```
```bash theme={null}
kubectl taint node node01 env_type=production:NoSchedule
```

Verify the taint:

```
```bash theme={null} kubectl describe node node01 | grep -i taint
```

#### ### Step 2: Create the Non-Tolerant Pod

Using an imperative command:

```
```bash theme={null}
kubectl run dev-redis --image=redis:alpine
```

### Step 3: Create the Tolerant Pod

Create a manifest (e.g., `question7.yaml`):

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: prod-redis spec: containers: - name: prod-redis
image: redis:alpine tolerations: - key: "env_type" operator: "Equal" value: "production" effect: "NoSchedule"
```

Apply it:

```
```bash theme={null}
kubectl apply -f question7.yaml
```

Finally, check that **prod-redis** is scheduled on **node01** while **dev-redis** is not:

```
```bash theme={null} kubectl get pod -o wide
```

\*\*\*

## Question 8 - Binding a PVC to a PV by Matching Access Modes

A PersistentVolumeClaim (\*\*app-pvc\*\*) in the \*\*storage-ns\*\* namespace is not binding with the PersistentV

After modifying the PVC manifest, remove the old PVC and apply the corrected file:

```
```bash theme={null}
kubectl delete pvc app-pvc -n storage-ns
kubectl apply -f <updated-pvc-manifest.yaml>
```

Verify the binding:

```
```bash theme={null} kubectl get pvc -n storage-ns
```

The PVC should now be **Bound** to the PV.

\*\*\*

## Question 9 - Troubleshooting a Faulty Kubeconfig File

The kubeconfig file **super.kubeconfig** (located at `~/root/CKA/super.kubeconfig`) is returning a "connect

```
```text theme={null}
https://controlplane:9999
```

Since the kube-apiserver listens on port **6443**, update the kubeconfig file as follows:

```
```yaml theme={null} clusters: - cluster: certificate-authority-data: server: https://controlplane:6443 name:
kubernetes
```

After saving the changes, test the connection:

```
```bash theme={null}
kubectl get node --kubeconfig=/root/CKA/super.kubeconfig
```

The connection should now work without errors.

## Question 10 – Scaling a Deployment

The **nginx-deploy** deployment currently has 1 replica. To scale it to 3 replicas:

1. Check the current status:

```
bash theme={null} kubectl get deployment nginx-deploy
```

1. Scale the deployment:

```
bash theme={null} kubectl scale deployment nginx-deploy --replicas=3
```

1. Verify the change:

```
bash theme={null} kubectl get deployment nginx-deploy
```

If the deployment still shows one available replica, review the deployment events:

```
```bash theme={null} kubectl describe deployment nginx-deploy
```

Troubleshoot any issues such as ReplicaSet misconfigurations or control plane component errors (for example, see [this](#)).

This completes Question 10.

\*\*\*

### ## Question 11 – Creating a Horizontal Pod Autoscaler (HPA) with Custom Metric

For the **api-deployment** in the **api** namespace, create an HPA that scales based on a custom pod metric.

Create a manifest (e.g., `question11.yaml`):

```
```yaml theme={null}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
  namespace: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-deployment
  minReplicas: 1
  maxReplicas: 20
  metrics:
  - type: Pods
    pods:
      metric:
        name: requests_per_second
      target:
        type: AverageValue
        averageValue: "1000"
```

Apply the HPA:

```
```bash theme={null} kubectl apply -f question11.yaml
```

Verify its configuration:

```
```bash theme={null}
kubectl describe hpa -n api
```

This completes Question 11.

---

## Question 12 – Configuring an HTTPRoute to Split Traffic

To distribute incoming web traffic, configure an HTTP route to split between **web-service** (80%) and **web-service-v2** (20%). The associated web gateway and services already exist.

Create an HTTPRoute manifest (e.g., `question12.yaml`):

```
```yaml theme={null} apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: web-route
namespace: default spec: parentRefs: - name: web-gateway namespace: default rules: - matches: - path: type:
PathPrefix value: / backendRefs: - name: web-service port: 80 weight: 80 - name: web-service-v2 port: 80
weight: 20
```

Apply the route:

```
```bash theme={null}
kubectl apply -f question12.yaml
```

This successfully routes 80% of traffic to **web-service** and 20% to **web-service-v2**.

---

## Question 13 – Upgrading an Application Using Helm

You need to upgrade an application using a Helm chart from the directory `/root/new-version`. Follow these steps:

### 1. Validate the Chart:

```
bash theme={null} helm lint /root/new-version
```

*Expected message:*

```
plaintext theme={null} ==> Linting /root/new-version [INFO] Chart.yaml: icon is recommended 1
chart(s) linted, 0 chart(s) failed
```

### 1. Install the Chart:

Use an auto-generated name:

```
bash theme={null} helm install --generate-name /root/new-version
```

List the releases:

```
bash theme={null} helm list
```

### 1. Uninstall the Old Version:

Replace `<old-release-name>` with the actual release name:

```
bash theme={null} helm uninstall <old-release-name>
```

Verify the installation:

```
```bash theme={null} helm list
```

This completes Question 13.

\*\*\*

### ## Question 14 - Outputting the Pod CIDR Network

To determine the pod CIDR network of the cluster and save it to `/root/pod-cidr.txt`, extract the podCIDR

```
```bash theme={null}
kubectl get node -o jsonpath='{.items[0].spec.podCIDR}' > /root/pod-cidr.txt
```

Verify the file content:

```
```bash theme={null} cat /root/pod-cidr.txt
```

\*Expected output (example):\*

```
```plaintext theme={null}
172.17.0.0/24
```

This completes Question 14 and wraps up the mock exam solutions.

---

End of Lesson.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## Whats Next

This course offers comprehensive mock exams for realistic CKA exam practice in a simulated environment.

Hello everyone, and welcome to the Ultimate CKA Mock Exam Series presented by Vijen Palazi from KodeKloud.

Before diving in, please ensure that you have completed all the prerequisite materials, including multiple mock exams and hands-on labs. If your CKA exam is approaching soon, it's essential to review all the background content first.

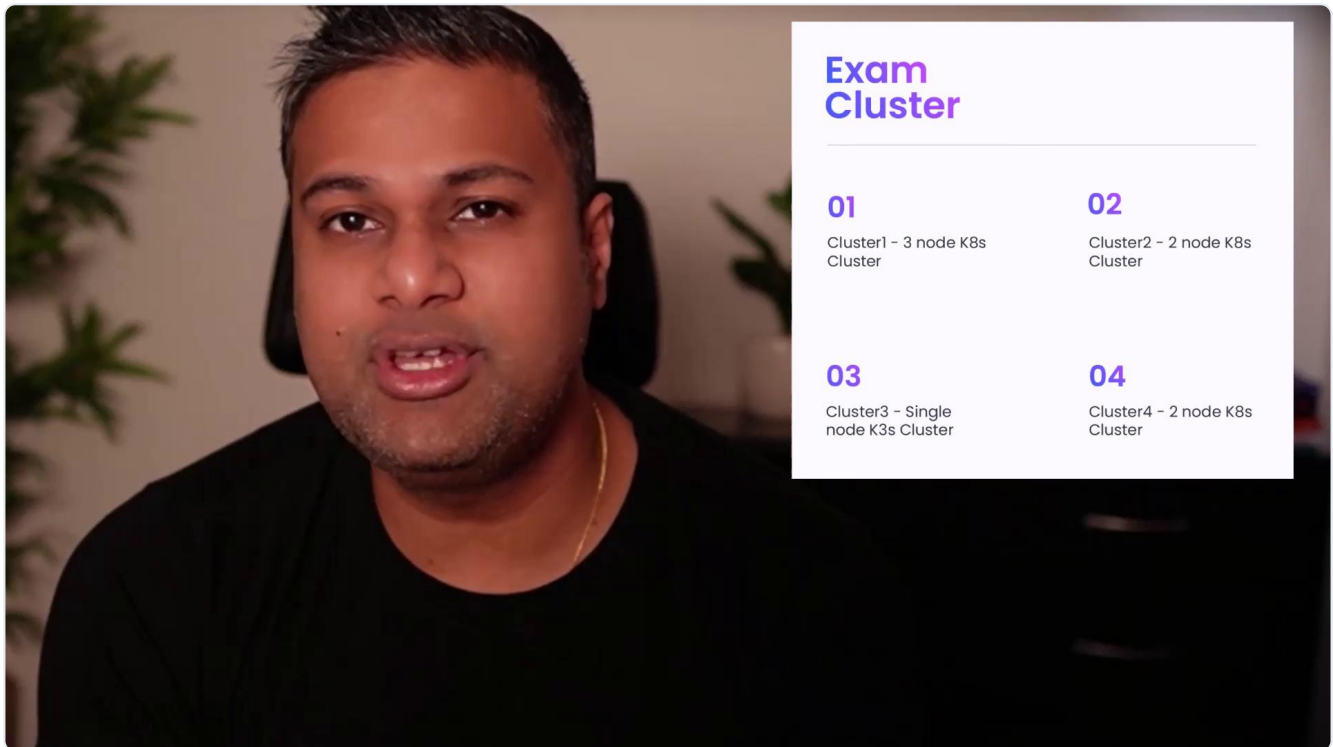
This course is designed as a series of comprehensive and challenging mock exams, offering you realistic, hands-on practice in a simulated exam environment. Unlike our regular labs, each mock exam uniquely mimics the actual test conditions.

The CKA exam assesses your practical knowledge in five key areas:

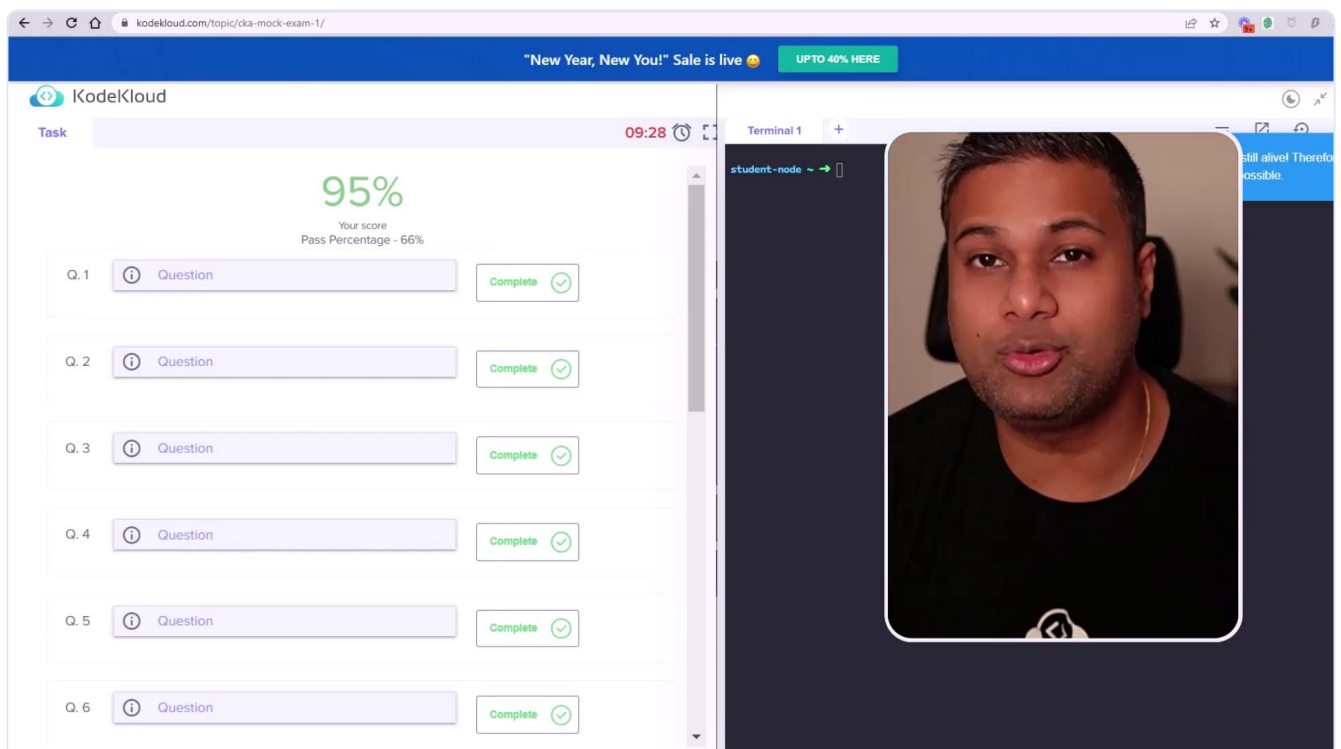
- Architecture, Installation, and Maintenance: 25%
- Workload Scheduling: 15%
- Service Networking: 20%
- Storage: 10%
- Troubleshooting: 30%



Each mock exam respects the weight of these areas, ensuring you receive a realistic testing experience. The series uses four Kubernetes clusters, with some clusters dedicated to specific knowledge areas. By default, you will be logged into the student node, giving you access to all other clusters and allowing SSH access to individual nodes.



This introduction marks the beginning of your journey through the Ultimate CKA Mock Exam Series.



Let's dive into one of these tests.

## Starting the Mock Exam

Click on a lab corresponding to one of the mock exams (for example, "CKA Mock Exam One"). The lab should load within 30 seconds—often in just one to two minutes.

Once loaded, you will see that the exam consists of 20 questions. The first question is from the Architecture, Installation, and Maintenance category, representing 25% of the exam. This question requires you to set the context to cluster three.

Before proceeding, verify all the configured clusters. Switch to cluster three with the following command:

```
```bash theme={null} kubectl config use-context cluster3
```

You are now logged into the student node. To view all configured clusters, run:

```
```bash theme={null}
kubectl config get-clusters
```

By default, without an explicitly set context, you connect to cluster one, which contains two worker nodes (node01 and node02). For this task, you must switch to cluster three. On the student node, execute:

```
```bash theme={null} kubectl config use-context cluster3 kubectl config get-clusters kubectl config get-contexts
kubectl get nodes
```

The student node serves as your client login node. If you open a new terminal session, it will also start

When your context is correctly set to cluster three, executing:

```
```bash theme={null}
kubectl get nodes
```

will display a single-node cluster consisting of the control plane (e.g., "cluster3-controlplane") running Kubernetes version 1.24.1. It is highly recommended to set the appropriate context for each question so you interact with the correct cluster.

For example, verify the nodes in cluster three with:

```
```bash theme={null} kubectl config use-context cluster3 kubectl get nodes
```

This command should return:

```
```bash theme={null}
NAME                STATUS    ROLES                  AGE     VERSION
cluster3-controlplane Ready    control-plane,master   18m     v1.24.1+k3s1
```

## Working Through a Question

Let's review a sample question. You may need to decode an existing secret, named "beta-sec-CK14-arc", which is created in a separate namespace. First, ensure the namespace exists by listing the secrets in that namespace:

```
```bash theme={null} kubectl get secret -n beta-ns-cka14-arch
```

Then, retrieve the secret in YAML format to inspect the data:

```
```bash theme={null}
kubectl get secrets -n beta-ns-cka14-arch -o yaml
```

The secret data is stored under the "data" section. To decode the secret from a base64-encoded string, run:

```
```bash theme={null} echo 'VGHpccyB0aG9lUGc2Vjcjmv0IQ=' | base64 -d
```

The decoded output should be: "This is the secret." You can redirect the decoded output directly into a file:

```
```bash theme={null}
kubectl config use-context cluster3
kubectl get secrets -n beta-ns-cka14-arch -o yaml

echo 'VGHpccyB0aG9lUGc2Vjcjmv0IQ=' | base64 -d > /opt/beta-sec-cka14-arch
```

Scroll through the exam interface to view the remaining questions. The first question is worth eight points from the Architecture, Installation, and Maintenance section. The second question from the same section can be attempted with:

```
```bash theme={null} kubectl config use-context cluster3 kubectl get secrets -n beta-ns-cka14-arch kubectl get
secrets -n beta-ns-cka14-arch -o yaml echo 'VGHpccyBpc28sIHlvdSBhcmUgY29tbWl0dGVklQ==' | base64 -d
echo 'VGHpccyBpc28sIHlvdSBhcmUgY29tbWl0dGVklQ==' | base64 -d > /opt/beta-sec-cka14-arch
```

Remember, you must compile all solutions before the allotted time expires. If time runs out, the exam will

<Callout icon="lightbulb" color="#1CB2FE">

Always double-check that you have switched the context appropriately using `kubectl config use-context` </Callout>

You might also encounter additional commands to review logs and secrets, such as:

```
```bash theme={null}
kubectl config use-context cluster1
kubectl logs -f color-app-cka13-arch
kubectl get secrets -n beta-ns-cka14-arch
kubectl get secrets -n beta-ns-cka14-arch -o yaml
echo 'VhgpcyBpyB0aGluZyBjc1QIQo=' | base64 -d > /opt/beta-sec-cka14-arch
```

At any time during the exam, you can click the "End Exam" button to check your score. In this demonstration, only the first question is attempted, with all other questions marked incorrect by default.

For example, reviewing the secret on the student node might show output like:

```
```bash theme={null} student-node → kubectl get secrets -n beta-ns-cka14-arch NAME TYPE DATA AGE beta-
sec-cka14-arch Opaque 1 2m19s
```

```
student-node → kubectl get secrets -n beta-ns-cka14-arch -o yaml apiVersion: v1 items: - apiVersion: v1 data:
secret: VGHpccyBpc8yB0aGluZyBQcmI0b3JpdHk= kind: Secret metadata: annotations: kubectl.kubernetes.io/last-
applied-configuration: | {"apiVersion":"v1","data":
{"secret":"VGHpccyBpc8yB0aGluZyBQcmI0b3JpdHk="},"kind":"Secret","metadata":{"name":"beta-sec-cka14-
```

```
arch","namespace":"beta-ns-cka14-arch"},"type":"Opaque"} creationTimestamp: "2023-02-02T17:08:47Z" name:
beta-sec-cka14-arch namespace: beta-ns-cka14-arch resourceVersion: "846" uid: ef9c5d98-
aec0-4d4d-99be-1cbb989b5e0 type: Opaque kind: List metadata: resourceVersion: "" selfLink: ""
```

student-node → echo VGhpcyBpc8yB0aGluZyBQcmI0b3JpdHk= | base64 -d This is the secret!

Additional exam commands might include:

```
```bash theme={null}
kubectl config use-context cluster1
kubectl get secrets -n beta-ns-cka14-arch -o yaml
curl http://cluster1-node01:30080
echo 'VGhpcyBpcyBhIHNLy3JldCE=' | base64 -d
echo 'VGhpcyBpcyB0aGUgc2VjcmV0IQ==' | base64 -d > /opt/beta-sec-cka14-arch
```

These steps ensure that you are working within the correct context and that your solutions are properly validated.

After completing your attempt, click the "End Exam" button to trigger automatic exam validation.

For reference, a sample terminal session may look like:

```
bash theme={null} student-node ~ → kubectl --context cluster1 create student-node ~ → kubectl --
context cluster1 create student-node ~ → kubectl --context cluster1 create student-node ~ →
kubectl --context cluster1 auth yes
```

Always verify your solutions and ensure you are working on the correct cluster context to avoid misconfigurations.

I hope you find this article useful. Best of luck with your CKA preparation!

Thank you.